CHAPTER 4: CPU SCHEDULING AND MULTIPROGRAMMING

4.1 Concept of CPU Scheduling

In a multiprogramming operating system, the CPU is switched among multiple processes to maximize processor utilization. When one process waits for an I/O operation to complete, the operating system takes the CPU away from that process and allocates it to another ready process. This scheduling mechanism ensures that the central processing unit does not remain idle.

4.2 The Process Scheduler and Queues

Processes enter the system and reside in the Ready Queue, waiting for allocation of CPU core time. The short-term scheduler (or CPU scheduler) selects a process from those in memory that are ready to execute and allocates the CPU to it.

4.3 Context Switching

When the CPU switches from executing process P1 to process P2, the kernel must save the state and context of P1 in its Process Control Block (PCB) and load the saved state of P2. This procedure is called a context switch. During a context switch, the CPU performs no productive user work; therefore, context-switch speed is critical to overall system throughput. State details saved include program counter, CPU registers, memory management registers, and I/O status information.

4.4 Scheduling Algorithms and Policies

4.4.1 First-Come, First-Served (FCFS) Scheduling

FCFS is the simplest non-preemptive CPU scheduling algorithm. The process that requests the CPU first is allocated the CPU first. While straightforward to implement with a FIFO queue, FCFS suffers from the 'convoy effect', where short processes get stuck waiting behind long, CPU-heavy processes, resulting in high average turnaround and waiting times.

4.4.2 Shortest-Job-First (SJF) Scheduling

SJF associates each process with the length of its next CPU burst. The process having the shortest expected CPU burst time is assigned the CPU first. SJF is provably optimal because it yields the minimum average waiting time for a given set of processes. Its preemptive variant is called Shortest-Remaining-Time-First (SRTF).

4.4.3 Round Robin (RR) Scheduling

Designed specifically for time-sharing systems, Round Robin defines a small unit of time called a time quantum (typically 10 to 100 milliseconds). The ready queue is treated as a circular queue. The scheduler traverses the queue, allocating the CPU to each process for a duration up to one time slice. If the process does not terminate within the time quantum, it is preempted and returned to the end of the ready queue.

4.5 Deadlocks and Synchronization

4.5.1 Deadlock Conditions

A deadlock occurs when two or more processes are permanently blocked because each holds a resource that the other requires. Four necessary conditions must hold simultaneously for a deadlock to arise:

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode.
2. Hold and Wait: A process must hold at least one resource and simultaneously wait to acquire other resources held by other processes.
3. No Preemption: Resources cannot be forcibly taken from a process holding them; they must be voluntarily released.
4. Circular Wait: A closed chain of processes exists such that each process holds resources requested by the next process in the chain.

4.5.2 Deadlock Prevention Methods

Deadlock prevention ensures that at least one of the four necessary conditions can never hold. For example, to prevent circular wait, the operating system can impose a total ordering on all resource types and require that every process requests resources in strictly increasing order of enumeration.